

Автоматизация проверки кода с помощью unittest

Цель занятия: научиться создавать автоматические модульные тесты, проверять функции и классы, контролировать исключения и обнаруживать регрессионные ошибки после изменения кода.

Продолжительность: лекция 30 минут, демонстрация 10 минут, практическая работа 45-55 минут, разбор 10 минут.

Результат обучения	Инструмент
Создаёт тестовые классы и методы	unittest.TestCase
Сравнивает результат с ожиданием	assertEqual и другие проверки
Проверяет исключения	assertRaises
Подготавливает данные перед тестом	setUp
Проверяет наследование	assertIsInstance и вызов методов

1. Зачем автоматизировать проверку

Ручная проверка подходит для знакомства с программой, но становится медленной, когда код изменяется много раз. Автоматический тест запускает сохранённый сценарий и сам сравнивает результат с ожидаемым значением.

Модульный тест проверяет небольшую самостоятельную часть программы: функцию, метод или класс. Набор таких тестов можно повторять после каждого рефакторинга.

Ручная проверка	Автоматический тест
Результат сравнивает человек	Результат сравнивает тестовая программа
Сценарий повторяется вручную	Набор запускается одной командой
Легко забыть одну проверку	Все сохранённые тесты повторяются
Удобна для исследования интерфейса	Удобен для функций и классов

2. Первый тест на unittest

Модуль unittest входит в стандартную библиотеку Python. Тесты обычно размещают в отдельном файле, например test_calculator.py.

```
import unittest

def add(a, b):
    return a + b

class TestAdd(unittest.TestCase):
    def test_positive_numbers(self):
        result = add(4, 6)
        self.assertEqual(result, 10)

if __name__ == "__main__":
    unittest.main()
```

Как устроен пример

1. import unittest подключает средства тестирования.
2. Класс наследуется от unittest.TestCase и получает методы проверки.
3. Название тестового метода начинается с test_.
4. assertEquals сравнивает фактический и ожидаемый результаты.
5. unittest.main() запускает тесты.

Результат успешного запуска

```
.
-----
Ran 1 test in 0.001s

OK
```

Точка означает успешный тест. Буква F означает проваленную проверку, а E - неожиданное исключение.

3. Структура Arrange Act Assert

```
def test_discount(self):
    # Arrange - подготовка
    price = 10000
    discount = 10

    # Act - действие
    result = calculate_price(price, discount)

    # Assert - проверка
    self.assertEqual(result, 9000)
```

4. Основные методы проверки

Метод	Назначение	Пример
<code>assertEqual(a, b)</code>	Значения равны	<code>self.assertEqual(result, 50)</code>
<code>assertNotEqual(a, b)</code>	Значения различаются	<code>self.assertNotEqual(result, 0)</code>
<code>assertTrue(value)</code>	Условие истинно	<code>self.assertTrue(student.is_active)</code>
<code>assertFalse(value)</code>	Условие ложно	<code>self.assertFalse(account.is_blocked)</code>
<code>assertIsNone(value)</code>	Получено None	<code>self.assertIsNone(result)</code>
<code>assertIsInstance(obj, type)</code>	Проверяется тип объекта	<code>self.assertIsInstance(user, Student)</code>
<code>assertIn(item, collection)</code>	Элемент присутствует	<code>self.assertIn("EKEB", title)</code>
<code>assertAlmostEqual(a, b)</code>	Приблизительное равенство	<code>self.assertAlmostEqual(result, 0.3)</code>

Для чисел с плавающей точкой применяют `assertAlmostEqual`, поскольку некоторые десятичные дроби представлены в памяти приближённо.

5. Один тест проверяет один сценарий

```
class TestExamResult(unittest.TestCase):
    def test_score_below_boundary(self):
        self.assertEqual(exam_result(49), "Незачёт")

    def test_score_on_boundary(self):
        self.assertEqual(exam_result(50), "Зачёт")

    def test_score_above_boundary(self):
        self.assertEqual(exam_result(51), "Зачёт")
```

Если разработчик заменит `>= 50` на `> 50`, тест значения 50 сразу покажет регрессионную ошибку.

6. Подготовка данных с помощью setUp

Метод setUp() вызывается перед каждым тестом. Каждый тест получает новый объект и не зависит от изменений, сделанных другим тестом.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def has_passed(self):
        return self.score >= 50

class TestStudent(unittest.TestCase):
    def setUp(self):
        self.student = Student("Алия", 50)

    def test_name(self):
        self.assertEqual(self.student.name, "Алия")

    def test_boundary_score(self):
        self.assertTrue(self.student.has_passed())
```

tearDown() вызывается после каждого теста. Он нужен для закрытия файлов, соединений и освобождения других ресурсов.

7. Проверка исключений

```
def calculate_average(total, count):
    if count == 0:
        raise ValueError("Количество не может быть равно нулю")
    return total / count

class TestAverage(unittest.TestCase):
    def test_zero_count(self):
        with self.assertRaises(ValueError):
            calculate_average(100, 0)
```

Тест пройдёт только при возникновении ожидаемого ValueError. Обычный результат или другой тип ошибки приведёт к провалу.

8. Тестирование наследования

После рефакторинга иерархии классов проверяют базовый и дочерний классы отдельно. Дочерний класс может наследовать неподходящее бизнес-правило.

```
class Student:
    def __init__(self, score):
        self.score = score
    def status(self):
        return "Зачёт" if self.score >= 50 else "Незачёт"

class GrantStudent(Student):
    def status(self):
        return "Грант сохранён" if self.score >= 70 else "Грант не сохранён"

class TestStudents(unittest.TestCase):
    def test_regular_boundary(self):
        self.assertEqual(Student(50).status(), "Зачёт")
    def test_grant_boundary(self):
        self.assertEqual(GrantStudent(70).status(), "Грант сохранён")
    def test_inheritance(self):
        self.assertIsInstance(GrantStudent(80), Student)
```

9. Требования к качественному тесту

- Независимость: тесты не зависят друг от друга.
- Повторяемость: одинаковые данные дают одинаковый результат.
- Понятное имя: название сообщает проверяемый сценарий.
- Одно ожидание: причина падения легко определяется.
- Известный результат: ожидание берут из требований.
- Границы: проверяют значение до границы, саму границу и значение после неё.

Ошибка	Последствие
Метод назван <code>check_result</code>	<code>unittest</code> не обнаружит его автоматически
Вместо <code>assert</code> используется <code>print</code>	Тест не определяет успех или ошибку
Все сценарии в одном методе	Трудно найти причину падения
Один изменяемый объект для всех тестов	Результат зависит от порядка запуска
Ожидание скопировано из программы	Общая ошибка кода и теста не обнаруживается

Автоматические тесты функций и классов

Цель: создать набор модульных тестов, найти дефекты, исправить программу и подтвердить исправление повторным запуском.

Порядок выполнения

1. Создайте файл `ekeb_service.py` с предложенным кодом.
2. Создайте `test_ekeb_service.py` и импортируйте проверяемые элементы.
3. Напишите тесты до исправления программы.
4. Запустите тесты и зафиксируйте ошибки.
5. Исправьте программный код.
6. Повторите весь набор и запишите итог.

Команды запуска

```
python -m unittest test_ekeb_service.py
python -m unittest discover
python -m unittest -v
```

Задание 1. Стоимость печати

Одна страница стоит 30 тенге; от 10 страниц включительно действует скидка 10 процентов; отрицательное количество вызывает `ValueError`.

```
def print_cost(pages):
    if pages < 0:
        return 0
    total = pages * 30
    if pages > 10:
        total = total * 0.9
    return total
```

Создайте тесты для 0, 1, 9, 10, 11 и отрицательного значения. Для ошибки примените `assertRaises`.

Задание 2. Результат экзамена

Допустимы баллы от 0 до 100. Значения вне диапазона вызывают ValueError. При 50 баллах и выше студент получает зачёт.

```
def exam_result(score):
    if score < 0 or score > 100:
        raise ValueError("Недопустимый балл")
    if score > 50:
        return "Зачёт"
    return "Незачёт"
```

Создайте отдельные тесты для -1, 0, 49, 50, 51, 100 и 101.

Задание 3. Класс Student

Объект хранит имя и балл. has_passed() возвращает bool. add_points() увеличивает балл, но итог не может превышать 100. Отрицательное количество вызывает ValueError.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
    def has_passed(self):
        return self.score > 50
    def add_points(self, points):
        self.score = self.score + points
        return self.score
```

- Используйте setUp() для Student("Алия", 50).
- Проверьте имя и начальный балл.
- Проверьте границу 50.
- Проверьте добавление 10 баллов.
- Проверьте максимум 100.
- Проверьте ValueError для отрицательных points.

Задание 4. Наследование

GrantStudent наследует Student. Для сохранения гранта требуется не менее 70 баллов. После рефакторинга переопределённый метод был удалён.

```
class GrantStudent(Student):  
    pass
```

- Проверьте, что GrantStudent является экземпляром Student.
- Проверьте статус при 69, 70 и 71 балле.
- Добавьте метод grant_status().
- Проверьте унаследованный add_points().
- После добавления баллов повторно проверьте грант.

Ожидаемые строки: «Грант сохранён» и «Грант не сохранён».

Задание 5. Регрессионная проверка

После исправления запустите полный набор тестов и заполните таблицу.

Показатель	Первый запуск	После исправления
Количество тестов		
Успешные тесты		
Ошибки F		
Ошибки E		
Итог OK или FAILED		

Описание одного дефекта

Поле	Ответ студента
Название дефекта	
Входные данные	
Ожидаемый результат	
Фактический результат	
Причина ошибки	
Подтверждение исправления	

Что нужно сдать

- ekeb_service.py с исправленным кодом.
- test_ekeb_service.py с тестами.
- Результат первого запуска.
- Успешный результат после исправления.
- Заполненную таблицу и описание дефекта.
- Вывод из 3-4 предложений.

Критерии оценивания

Критерий	Баллы
Корректный тестовый класс и методы test_*	2
Обычные и граничные значения	2
Применены setUp и assertRaises	2
Проверены Student и GrantStudent	2
Код исправлен и полный набор проходит	2

Дополнительное задание

Создайте ExcellentStudent, наследующий Student. scholarship() возвращает 30000 при 90-100 баллах, 15000 при 75-89 и 0 в остальных случаях. Проверьте 74, 75, 89, 90 и 100.

Вопросы для защиты

1. Почему имя метода начинается с test_?
2. Чем assertEquals отличается от print()?
3. Зачем setUp создаёт объект перед каждым тестом?
4. Как проверить ожидаемое исключение?
5. Какие значения проверяют условие score >= 50?
6. Почему после исправления запускают весь набор?